Flags

Computer Organization & Assembly Language

Flags Register

This is a special *register* in every architecture called the *flags register* or the *program status word*.

It is meaningless as a unit, rather the individual *bits* carry different meanings.

The *bits* of the accumulator work in parallel as a unit and each *bit* mean the same thing.

The *bits* of the flags register work independently and individually, and combined its value is meaningless.

The individual *bits* are automatically set or cleared as side effects of arithmetic and logical operations.

From FLAGS to EFLAGS to RFLAGS

Like the general-purpose *registers*, the *flags register* has evolved alongside the x86 architecture, growing wider with each generation while preserving every bit from the previous generation.

In the Intel 8086, the *register* is 16 bits wide. Its successors, the *EFLAGS* and *RFLAGS registers* in modern x86-64, are 32 bits and 64 bits wide respectively. The wider registers retain compatibility with their smaller predecessors.

In practice, only the lower 32 bits of *RFLAGS* contain meaningful *flags*. The upper 32 bits are reserved and must remain zero for compatibility. This means *EFLAGS* and the lower half of RFLAGS are functionally identical, and in most programming contexts you will see references to EFLAGS even on x86-64 systems.

Categories of Flags

The *flags* in the *register* are divided into three main categories:

Status flags that indicate outcomes of arithmetic, logical, and comparison instructions

Control flags that manage execution behavior

System flags that handle privilege levels, virtualisation, and other advanced features.

The RFLAGS Register Layout

The RFLAGS register bits and their descriptions are as follows:

bit 0 is CF (Carry *Flag*),

bit 2 is PF (Parity *Flag*),

bit 4 is AF (Auxiliary Carry *Flag*),

bit 6 is ZF (Zero *Flag*),

bit 7 is SF (Sign *Flag*),

bit 8 is TF (Trap *Flag*),

bit 9 is IF (Interrupt Enable *Flag*),

bit 10 is DF (Direction *Flag*),

bit 11 is OF (Overflow *Flag*),

bits 12–13 are IOPL (I/O Privilege Level),

bit 18 is AC (Alignment Check), and

bit 21 is ID (Identification).

The RFLAGS Register Layout

Bit:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flag:	0	NT	IOPL	IOPL	OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

Bits 1, 3, 5 have fixed values (1, 0, 0). *Bit* 15 is always 0.
Bits 22 to 29 and bit 32 through 63 of RFLAGS are currently **reserved** and are generally set to zero.
Bit 30 - AES key schedule loaded flag (none): 0
Bit 31 - Alternate Instruction Set (AI): 0

The Status Flags

The Zero Flag (ZF) — Bit 6

The *Zero Flag* is set to 1 when the result of an operation is exactly zero across all bits, and cleared to 0 when the result is anything other than zero.

It can be used by the ***jz***(jump if last result was zero) or ***jnz*** instructions. ***je*** (jump if equal) and ***jne*** (jump if not equal) are just aliases of ***jz*** and ***jnz***, because if the difference is zero, then the two values are equal.

When we write CMP EAX, EBX followed by **JE** label, the processor does not literally have a "jump if equal" capability in hardware.

The Status Flags

The Zero Flag (ZF) — Bit 6

What it does is subtract EBX from EAX (without storing the result), check whether that subtraction produced zero, and jump if **ZF** = 1. Equality and zero are the same thing from the processor's perspective two values are equal precisely when their difference is zero.

Operation	Result	ZF
`SUB EAX, EAX`	EAX − EAX = 0	1
`ADD EAX, 0`	EAX + 0 = EAX (if EAX ≠ 0)	0
`XOR EAX, EAX`	EAX ⊕ EAX = 0 (all bits cancel)	1
`MOV EAX, 5; SUB EAX, 5`	5 − 5 = 0	1
`AND EAX, 0`	EAX AND 0 = 0	1

The Status Flags

The Sign Flag (SF) — Bit 7

The *Sign Flag* reflects the sign of the result when that result is interpreted as a *signed* (two's complement) number. It is set to 1 when the most significant bit (MSB) of the result is 1 which in two's complement representation means the result is *negative* and cleared to 0 when the MSB is 0, meaning the result is *non-negative.*

Mathematically, *SF* is set to 1 if and only if the result was negative. Physically, *SF* is set to 1 if and only if the most significant bit of the result is 1.

For unsigned arithmetic, the MSB is just a data bit and *SF* is largely irrelevant. For signed arithmetic, *SF* directly tells you the sign of the result.

The Status Flags

The Sign Flag (SF) — Bit 7

Setting conditions for SF:

Operation (32-bit)	Result (hex)	MSB	SF
`MOV EAX, 5; SUB EAX, 3`	0x00000002 (+2)	0	0
`MOV EAX, 3; SUB EAX, 5`	0xFFFFFFFE (−2 in two's complement)	1	1
`MOV EAX, 0x80000000`	0x80000000 (most negative 32-bit signed)	1	1
`ADD EAX, 1` (EAX was 0x7FFFFFFF)	0x80000000	1	1

The Status Flags

The Carry Flag (CF) — Bit 0

The *Carry* Flag is the flag associated with *unsigned arithmetic overflow.* It records whether an arithmetic operation produced a carry out of or a borrow into the most significant bit position of the result.

For addition, *CF* is set when the result exceeds the maximum value representable in the operand size. For subtraction, CF is set when the result would be negative in unsigned interpretation that is, when you are subtracting a larger number from a smaller one.

CF contains the bit that carries out of an addition or subtraction. For signed numbers, this does not really indicate a problem, but for unsigned numbers, this indicates an overflow.

The Status Flags

The Carry Flag (CF) — Bit 0

Addition example

```
  1111 1111          (0xFF = 255 unsigned)
+ 0000 0001          (0x01 = 1 unsigned)
-------------
1 0000 0000          (0x100 = 256, but only 8 bits fit → result = 0x00)
↑
```

Carry out → CF = 1

The result in the 8-bit destination is 0x00, which is wrong for unsigned arithmetic (255 + 1 should be 256).

CF = 1 signals that the true result did not fit.

The Status Flags

The Carry Flag (CF) — Bit 0

Subtraction example unsigned:

0000 0101 (5 unsigned)

\- 0000 1010 (10 unsigned)

1111 1011 (0xFB = 251 unsigned, which is wrong should be negative)

↑

Borrow required → CF = 1

The Status Flags

The Overflow Flag (OF) — Bit 11

The *Overflow* Flag is the flag associated with *signed arithmetic overflow.* It is set when an arithmetic operation produces a result that cannot be correctly represented as a signed two's complement number in the given operand size that is, when the mathematical result falls outside the range

$[-2^{(n-1)} , 2^{(n-1)-1}]$ for an n-bit operand.

The classic rule for detecting signed *overflow*: *overflow* occurs when two operands of the same sign are added and the result has the opposite sign, or when two operands of opposite sign are subtracted and the result has the same sign as the subtrahend.

OF = 1 means *the sign bit of the result is wrong* the carry into the sign bit and the carry out of the sign bit differ.

The Status Flags

The Overflow Flag (OF) — Bit 11

Positive overflow: adding two positive numbers gives a negative result

0111 1111 (+127, maximum positive 8-bit signed)

\+ 0000 0001 (+1)

1000 0000 (−128 in two's complement, wrong sign!)

OF = 1

Negative overflow: adding two negative numbers gives a positive result

1000 0000 (−128)

\+ 1111 1111 (−1)

---

1 0111 1111

(+127 — wrong sign! carry out = 1, carry into sign bit = 1, equal → OF = 0)

The Status Flags

The Overflow Flag (OF) — Bit 11

127 + 1 = 128

(overflow in signed 8-bit):

Carry into bit 7: 1

(from bit 6 addition 1+1+1 = 11, carry = 1)

Carry out of bit 7: 0

(1+0+1 = 10, write 0, carry = 1... let us just state the rule)

Result = 0x80 = -128

(wrong sign for positive + positive)

OF = 1

; −128 + (−1) = −129

(overflow in signed 8-bit):

0x80 + 0xFF = 0x17F, take low 8 bits:

0x7F = +127 (wrong sign)

OF = 1

; 50 + 70 = 120

(no overflow, fits in signed 8-bit range [−128, 127]):

Result = 0x78 = +120 (correct sign)

OF = 0

The Status Flags

The Overflow Flag (OF) — Bit 11

The critical distinction between CF and OF:

CF is for unsigned arithmetic (it means the unsigned result does not fit).

OF is for signed arithmetic (it means the signed result has the wrong sign).

The same bit pattern after the same addition can set **CF** without setting **OF**, or **OF** without **CF**, or both, or neither, depending on how you interpret the operands.

This is why x86 maintains both flags independently, different conditional jump instructions check different flags depending on whether the comparison is for signed or unsigned values.

The Status Flags

The Parity Flag (PF) — Bit 2

The Parity Flag is set to 1 when the number of set bits (bits equal to 1) in the *lowest byte* of the result is even, and cleared to 0 when the count of set bits in the lowest byte is odd. This is called *even parity*.

PF and AF are really unusual ancient flags, holdovers from the 8-bit days.

PF returns odd parity, like for serial communication.

The Status Flags

The Parity Flag (PF) — Bit 2

PF was historically important for error detection in serial communication verifying that a transmitted byte had the expected parity. In modern *programming* it is rarely used directly, but it is still set by most arithmetic and logical instructions and can be tested with JP (jump if parity, i.e., even parity, PF = 1) and JNP (jump if no parity, PF = 0).

Example:

Result = 0xA3 = 1010 0011b. Set bits: bits 0, 1, 5, 7 → 4 set bits (even). PF = 1.

The Status Flags

The Auxiliary Carry Flag (AF) — Bit 4

The *Auxiliary Carry Flag* records a carry out of bit 3 into bit 4 — a carry from the low nibble (lower 4 bits) to the high nibble (upper 4 bits) of the lowest byte of the result. It was designed for *Binary Coded Decimal (BCD)* arithmetic, where digits 0–9 are each stored in a 4-bit nibble, and an overflow of a nibble means a carry to the next decimal digit.

BCD arithmetic and AF are virtually unused in modern software and x86-64 has actually removed BCD adjustment instructions in 64-bit mode. However, **AF** is still set by arithmetic instructions and we may encounter it in legacy code or in older reference materials

Instructions That Set the Flags

Not every instruction sets the flags.

MOV, for example, does not affect flags at all. Knowing which instructions set flags and which do not is essential for writing correct assembly, because a careless MOV between a comparison and a conditional jump will silently destroy the flag state.

Instructions That Set All Four Main Condition Codes (CF, ZF, SF, OF)

The arithmetic and logical instructions that produce results and update all four condition codes are the core of flag-setting behavior.

Instructions That Set the Flags

ADD and SUB:

Set CF, ZF, SF, OF, PF, AF based on the result of the addition or subtraction.

AND, OR, XOR:

Set ZF and SF based on the result.

CF and OF are always *cleared to 0* after logical operations

PF and AF may be set.

INC and DEC:

Increment and decrement instructions update ZF, SF, OF, PF, and AF but deliberately *do not affect CF.*

Instructions That Set the Flags

NEG (Negate):

Performs two's complement negation. Sets CF = 0 if the operand was zero, CF = 1 otherwise (because negating any non-zero value is equivalent to subtracting from zero, which borrows).

Sets ZF if result is zero, SF based on sign bit, OF if operand was the most negative value (negating −128 in 8-bit gives −128, which overflows).

Instructions That Set the Flags

CMP and TEST: The Comparison Instructions

These two instructions are the workhorses of flag-setting before conditional jumps.

They are unique in that they *perform an operation solely to set flags, discarding the result.*

Instructions That Set the Flags

CMP (Compare):

CMP destination, source

performs destination − source and sets all flags based on the result, but *does not* store the difference anywhere. The destination register is completely unchanged.

The cmp instruction tells the subsequent conditional jump about the comparison via the EFLAGS register. cmp eax, 10 actually internally subtracts 10 from the value in EAX. If the difference is zero, the CPU sets ZF. sub eax, 10 actually sets all the same flags — so two numbers can be compared with cmp A, B or sub A, B and get the same flag results, but they are not totally interchangeable: cmp will not change A.

Instructions That Set the Flags

CMP (Compare):

CMP RAX, RBX

Computes RAX - RBX, sets CF/ZF/SF/OF, discards result

RAX and RBX are both unchanged

Instructions That Set the Flags

TEST (Bit Test):

TEST destination, source

performs destination AND source and sets ZF, SF, PF based on the result, always clears CF and OF, but does *not* store the AND result. It is most commonly used in two specific patterns:

Testing whether a register is zero: TEST RAX, RAX ANDs the register with itself if the register is zero, every bit ANDs to zero, and ZF = 1. This is how compilers check for NULL pointers or zero loop counters.

Instructions That Set the Flags

TEST (Bit Test):

Testing whether a specific bit is set: TEST RAX, 1 ANDs RAX with 1, which isolates bit 0.

If bit 0 of RAX is 0, result is 0 and ZF = 1.

If bit 0 is 1, result is 1 and ZF = 0.

```
TEST RAX, RAX        ; if RAX is zero → ZF = 1; if RAX is non-zero → ZF = 0
TEST RCX, 1          ; test bit 0 of RCX (check if RCX is odd)
```

Control Flags

The Direction Flag (DF) — Bit 10

The *Direction Flag*, located at bit 10 of EFLAGS, determines the addressing direction for string-processing instructions like MOVS, CMPS, SCAS, LODS, and STOS. When **DF** is cleared to 0, the index registers RSI and RDI increment after each operation, processing data forward. When set to 1, they decrement, processing data backward.

The instructions for explicitly setting DF are:

CLD — Clear Direction Flag (DF ← 0), meaning strings are processed forward (low address to high address)

STD — Set Direction Flag (DF ← 1), meaning strings are processed backward

Control Flags

The Interrupt Flag (IF) — Bit 9

When the Interrupt Flag is set, the processor will ignore maskable interrupts. *Non-maskable interrupts* are uninhibited by this flag and will execute regardless.

When IF = 1, *maskable interrupts are enabled*. When IF = 0, maskable interrupts are disabled (the processor ignores them). This is controlled by:

STI — Set Interrupt Flag (IF ← 1): enables maskable interrupts

CLI — Clear Interrupt Flag (IF ← 0): disables maskable interrupts

Control Flags

The Interrupt Flag (IF) — Bit 9

These *instructions* are *privileged* they can only be executed in kernel mode (ring 0).

An *application program* trying to execute CLI or STI will cause a general protection fault.

Operating system kernels use CLI/STI to protect *critical sections* of code (like interrupt handlers themselves) where an *interrupt* arriving in the middle of the code would corrupt *data structures*.

Control Flags

The Trap Flag (TF) — Bit 8

The *Trap Flag* is a *debug flag*.

Setting it will put the processor into *single-step* mode: after each instruction is executed it will call the *Single Step* Interrupt *Handler*, which is expected to be handled by a *debugger*.

When TF = 1, the CPU generates a *debug* exception (interrupt 1) after executing each instruction. The *debugger's* interrupt handler catches this, displays register state, and waits for the user to press a key.

This is the mechanism behind the step-by-step execution feature in every *debugger.*